

UC MATH 600 MATHEMATICS FOR COMPUTER SCIENCE · MODULE 04 · LECTURE NOTES

Sets, Functions, Relations

The vocabulary every later course assumes you have.

MODULE OUTCOMES

- Work with sets, operations, and power sets.
- Define functions precisely, and classify them.
- Use relations, equivalence, and partial orders.
- Compare infinite sets by bijection.
- Explain the diagonal argument.

1 Sets

- **Membership, subset, union, intersection, difference, and complement** — and the distinction between $\in$ and $\subseteq$, which is the first trap: **a set can be both an element of and a subset of the same thing**, and confusing the two produces statements that are not merely false but meaningless.
- **The power set of an n -element set has 2^n elements** — which is the counting fact used most often in **this program**, from CSCE 637's counting argument for circuit lower bounds to CSCE 640's amplitude count to CSCE 717's valuation functions.
- **Cartesian products and tuples, which is how every relation (§3) and every graph edge (Module 07) is formally defined**, and is worth being comfortable with even though it is rarely written out.
- **And the empty set is a subset of every set**, which is the second trap — **vacuously true statements are true** ('every element of the empty set is purple' is true) — **and a proof that forgets the empty case is incomplete**, which is a common and very quiet error.
- **Plus: a set has no order and no repetition**, so $\{1,2\} = \{2,1\} = \{1,1,2\}$ — **and if you need order or repetition, you wanted a sequence or a multiset** and should say so.

2 Functions

A FUNCTION ASSIGNS EXACTLY ONE OUTPUT TO EVERY INPUT OF ITS DOMAIN — AND BOTH HALVES ARE CHECKABLE CONDITIONS

Every element of the domain gets an output (totality), and **exactly one output** (well-definedness) — **and both of these are things that can actually fail**, especially when you define a function by a formula applied to equivalence classes or by a procedure that might not terminate.

Injective (one-to-one): different inputs give different outputs. Surjective (onto): every element of the codomain is hit by something. Bijective: both — and only a bijection has an inverse function, which is why §4's size definition uses bijections specifically.

Which is the vocabulary CSCE 627 and CSCE 637 use without stopping to define, and which §4 needs immediately.

And 'well-defined' is a real proof obligation rather than a pleasantry: if you define f on an equivalence class by picking a representative and computing with it, you must show the answer does not depend on which representative you picked — a step people routinely skip, and one of the places modular arithmetic (Module 05) requires care.

3 Relations

Property	Meaning	Where it shows up
Reflexive	Every element relates to itself.	Both equivalences and partial orders.
Symmetric	If $a \sim b$ then $b \sim a$.	Equivalence relations.
Antisymmetric	$a \leq b$ and $b \leq a$ together force $a = b$.	Partial orders — and note this is not the negation of symmetric.
Transitive	$a \sim b$ and $b \sim c$ give $a \sim c$.	Both. This is the one most often assumed without checking.
Equivalence relation	Reflexive + symmetric + transitive.	Partitions the set into disjoint classes — see the note.
Partial order	Reflexive + antisymmetric + transitive.	Scheduling and dependency graphs, type systems, lattices — and CSCE 605 uses them heavily.

An equivalence relation and a partition are the same thing — every equivalence relation splits its set into disjoint classes, and every partition into disjoint classes defines an equivalence relation — which is worth proving once and then using freely. **The equivalence-equals-partition correspondence is the fact to carry forward:** modular arithmetic (Module 05) is entirely an application of it.

4 Infinite sets

TWO SETS HAVE THE SAME SIZE IF A BIJECTION EXISTS BETWEEN THEM — AND THIS GIVES SURPRISING ANSWERS

The integers and the even integers have the same size, by the bijection $n \rightarrow 2n$ — **which offends intuition, and is a consequence of the definition rather than a paradox.** For infinite sets, a proper subset can be the same size as the whole; that is what infinite means.

And the rationals are countable too, by a zigzag enumeration of the grid of numerators and denominators — so 'more densely packed' does not mean 'bigger' either.

But the reals are not countable — Cantor's diagonal argument: given any proposed list of all reals, construct a new one that differs from the first in its first digit, from the second in its second, and so on, **so no list can be complete.**

Which is the single most reused argument in theoretical computer science — CSCE 627's undecidability result and CSCE 637's hierarchy theorems are both diagonal arguments — and recognising the shape is worth more than remembering the result.

AND THE COUNTING CONSEQUENCE THAT MATTERS FOR COMPUTING

There are countably many programs — each one is a finite string over a finite alphabet, and the set of finite strings is countable — **and there are uncountably many functions from the naturals to $\{0,1\}$** , by the diagonal argument.

So almost every function is not computable by any program, by counting alone — before any halting-problem argument, and without any model of computation beyond 'programs are finite strings'.

Which is CSCE 627's opening result, and it follows from this module rather than from anything about computation, which is a surprising place for it to come from.

And it is a good demonstration of what counting arguments buy you: a sweeping conclusion from essentially no machinery — a pattern worth looking for, and one Module 06 develops properly.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lehman, Leighton & Meyer, chapters 4 and 8 (free)	§§1 to 4, free — and the infinite-sets chapter is unusually careful.
Velleman, chapters 4 and 5	§§2 and 3 — relations and functions with the well-definedness obligation made explicit. Library copy.
Hammack — Book of Proof, chapters 11 and 14 (free)	§§3 and 4, free — relations, and cardinality with the diagonal argument worked slowly.
Sipser, chapter 4 (library copy)	§4's consequence — where the diagonal argument becomes undecidability, which is CSCE 627.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Give a set that is both an element and a subset** of another.
- 2 **Prove the power set has 2^n elements**, by induction.
- 3 **Find a statement that is vacuously true** and explain why it is true.
- 4 **Give a formula that fails to be well-defined** on equivalence classes.
- 5 **Classify five functions** as injective, surjective, or bijective.
- 6 **Check four relations** against all four properties.

- 7 **Prove the equivalence-partition correspondence**, both directions.
- 8 **Give a bijection** between the naturals and the integers.
- 9 **Write out the diagonal argument** in full.
- 10 **Count the programs and the functions**, and state the conclusion.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish $\in$ and $\subseteq$ with an example.
- 2 How many subsets does an n -element set have, and why?
- 3 Why are vacuously true statements true, and what does that mean for proofs?
- 4 Give the two conditions a function must satisfy.
- 5 Define injective, surjective, bijective, and say which has an inverse.
- 6 What is the well-definedness obligation?
- 7 Name four relation properties and the two combinations.
- 8 State the equivalence-partition correspondence.
- 9 How are two infinite sets compared, and give a surprising consequence.
- 10 State the diagonal argument and its counting consequence for programs.